novelty at lower k (e.g., $k = 50$), under a setting where the candidate model only adds k additional items to be sent to the downstream ranking layer. While our definition assumes a single base model, it can be easily extended to multiple deployed models by pooling their top-L predictions for novelty comparison.

Note that novelty of top-k items of a candidate algorithm is different from its diversity [31], which measures the distance of items *within* the top-k list produced by the candidate algorithm.

3.2 RL formulation using policy gradient

`Novelty@k`, as defined above, cannot be optimized directly since it includes a non-differentiable component w.r.t. the encoder's parameters, the top-k sorting operation over items. As in past work on recommendation systems [16, 33, 38], a natural solution for non-differentiable rewards is to use reinforcement learning. Formally, our problem can be formulated as a one-step RL setting. The query is considered as the *state* and the set of top-k items as the *action*. A policy $\pi : X \rightarrow \{Z_k : Z_k \subseteq Z\}$ is a function that outputs a set of top-k items given a query. For each action selected by the policy, the environment provides reward feedback on the < state,action > pair. For instance, in the query-ad recommendation task, user's input query is the state, the top-k predicted ads are the action by the policy, and the environment provides reward based on the query and its top-k predicted ads, e.g., based on the novelty and relevance of the predicted ads. Given a reward function $\mathcal{R}$, the task to learn a policy π_θ (parameterized by θ), that optimizes,

$$\max_\theta \mathbf{E}_{x\sim\mathcal{D}}\mathbf{E}_{\mathbf{y}\sim\pi_\theta(x)}\mathcal{R}(x,\mathbf{y}) \tag{2}$$

In our work, we consider a pre-trained encoder $f_\theta : X \cup Z \rightarrow \mathbf{R}^d$ to correspond to the initial policy π^0_θ. For simplicity, given an encoder/policy at any stage of training, the top-k items $\mathbf{y}$ are assumed to be independently sampled from the discrete action probability distribution, $\pi_\theta : X \rightarrow Z;\ y_j \sim \pi_\theta(z|x)\forall j \in \{1, 2, 3..k\}$; where $\pi_\theta(z|x) = \text{softmax}_Z\ \texttt{sim}(f(x), f(z))$. We use $\pi_\theta(\mathbf{y}|x)$ to denote the top-k items generated using the π_θ policy.

As in past work on using RL in recommendation systems [2, 24, 27], we use a policy gradient algorithm to optimize the novelty reward. Policy gradient algorithms are well-suited for recommender systems since they do not explicitly parameterize the action space (thus allowing a large number of actions) and can directly update the parameters of the encoder f_θ. Specifically, we use the REINFORCE [36] algorithm that depends on a Monte Carlo approximation of the reward expectation from Eqn. 2.

$$\mathbf{E}_{x\sim\mathcal{D}}\mathbf{E}_{\mathbf{y}\sim\pi_\theta(x)}\mathcal{R}(x,\mathbf{y}) \approx \frac{1}{B}\sum_{i=1}^{B}\mathcal{R}(x^{(i)},\mathbf{y}^{(i)}) \tag{3}$$

where $y_j^{(i)} \sim \pi_\theta(x^{(i)})\forall j \in \{1, 2, 3, \cdots, k\}$ and B is the batch size. The loss gradient is given by,

$$\begin{aligned}\nabla\mathbf{E}_{x\sim\mathcal{D}}\mathbf{E}_{\mathbf{y}\sim\pi_\theta(x)}\mathcal{R}(x,\mathbf{y}) &= \mathbf{E}_{x\sim\mathcal{D}}\mathbf{E}_{\mathbf{y}\sim\pi_\theta(x)}\mathcal{R}(x,\mathbf{y})\nabla\log\pi_\theta(\mathbf{y}|x)\\ &\approx \frac{1}{B}\sum_{i=1}^{B}\mathcal{R}(x^{(i)},\mathbf{y}^{(i)})\nabla\log\pi_\theta(\mathbf{y}^{(i)}|x^{(i)})\end{aligned} \tag{4}$$

where $y_j^{(i)} \sim \pi_\theta(x^{(i)})\forall j \in \{1, 2, 3, \cdots, k\}$. Since the reward is one-step, the above optimization can be interpreted to have a simple goal: increase the probability of items that occur in a k-sequence with high reward, and decrease the probability of items that occur in a k-sequence that obtains low reward. Note that we use the REINFORCE algorithm for simplicity, but any other policy gradient algorithm can be used.

3.3 LLMs make novelty optimization practical

While the formulation is reasonable, there is a key limitation: novel items, by definition, do not have any user feedback data since they were never shown to users by the base production model for that query. Hence, if we use only the log-based training data for Eqn. 4, relevance feedback for the novel items would be missing and thus no novel item's reward can be computed.

In this paper, we show that LLMs help avoid this limitation by providing relevance feedback for novel query-item pairs. LLMs such as GPT-3.5 and GPT-4 have been shown to provide a substantial improvement in semantic understanding compared to existing encoder-based models. For instance, recent work shows that GPT-3.5 can match or surpass crowd-sourced human labellers in their accuracy on labelling [10] or ranking [12] retrieval outputs for relevance. In addition, even though the relevance criterion for a correct recommendation may differ for different tasks, we need not train separate relevance models. For example, while the criteria for matching books to other books may be different from matching queries to ads, which in turn may be slightly different for matching queries to webpages; a single LLM like GPT-3.5 can be used for providing relevance feedback for all these domains by simply changing the prompt. Hence, the accuracy and universality of LLMs make it possible to obtain relevance feedback for arbitrary query-item pairs and optimize novelty directly for retrieval tasks.

3.4 The challenge with large action spaces

While LLMs provide a solution to the problem of relevance feedback for novel query-item pairs, another key challenge is the large number of potential query-item pairs to label since available items are typically in the millions. Below we show that the number of relevance reward samples needed for a policy gradient algorithm to obtain a desired accuracy (*sample complexity*) increases proportional to the square of the number of actions. The result is based on applying finite sample convergence bounds [23] for policy gradient algorithms to the retrieval setup.

Let π^* denote the optimal policy that maximizes Eqn 2 and t refer to the steps of the optimization. We consider a one-step Markov Decision Process where each query corresponds to a state and the actions are the top-k predicted items. Hence, for each state, we only need to optimize the current reward. We assume a uniform probability for sampling each state (query).

Assumption 1 (from [23])**.** *(Sufficient exploration). The initial state μ distribution satisfies* $\min_s \mu(s) > 0$.

Assumption 2. *For each state, the initial policy's probability for selecting the optimal action is better than random (uniform probability) within some multiplicative constant $\rho \geq 1$: $\pi_{\theta_0}(\mathbf{y}^*(x)|x) > \frac{1}{\rho A}\ \forall x \in D$ where $\mathbf{y}^*(x) := \arg\max_{\mathbf{y}} \pi^*(\mathbf{y}|x)$.*

Proposition 1. *Let Assumptions 1 and 2 hold and let $\{\theta_t\}_{t\geq 1}$ be generated using the standard policy gradient update: $\theta_{t+1} \leftarrow$*

$\theta_t + \eta \frac{\partial V^{\pi_{\theta_t}}(\mu)}{\partial \theta_t}$ *where* $V^{\pi_{\theta_t}}(\mu) = \mathbf{E}_{x\sim\mathcal{D}}\mathbf{E}_{\mathbf{y}\sim\pi_{\theta_t}(.|x)}\mathcal{R}(x,\mathbf{y})$. *Then, for all* $t \geq 1$, *with* $\eta = 1/8$,

$$\mathbf{E}_{x\sim\mathcal{D}}[(\pi^*(x) - \pi_{\theta_t}(x))^T \mathbf{r}(x)] \leq \frac{16SA^2\rho^2}{t} \left\| \frac{1}{\mu} \right\|_\infty \tag{5}$$

where $S = |X|$ *is the number of states and* A *is the number of actions.*

Proof is in Appendix. The proof uses Thm. 4 from [23] for a Markov Decision Process and adapts it to the single-step problem and additionally uses Assumption 2 to express the bound in terms of ρ and A. Since the policy outputs a list of top-k items, the total number of possible actions is $A = C(|Z|, k)$, indicating a high sample complexity. In the next section, we describe how we reduce the effective sample complexity and derive a practical algorithm.

4 LARGE ACTION POLICY GRADIENT

Proposition 1 shows that a naive application of the policy gradient algorithm will be slow to converge to the optimal reward-maximizing solution due to a combinatorially large action space. First, we show that the combinatorial action space $A = C(|Z|, k)$ can be decomposed into item-wise action space, $A = |Z|$ whenever the reward over top-k items can be decomposed as an additive sum over item-wise rewards. We show that most common novelty reward functions satisfy this property. Second, we provide a reformulation of the RL problem that reduces the action space to a binary decision and further increases the rate of convergence.

4.1 Reduction from Top-k to item-wise rewards

As stated in Section 3, the reward function is a combination of novelty and relevance rewards. We assume that both novelty and relevance rewards compose additively. That is, given a query x, $Novelty@k(x, \phi, \psi, L) = \sum_{j=1}^{k} Novelty(\langle x, z_j\rangle, \phi, \psi, L)$, and we have $Relevance@k(x, \phi) = \sum_{j=1}^{k} Relevance(\langle x, z_j\rangle, \phi)$; where z_j are the individual item predictions and $Relevance@k$ is typically recall@k or precision@k. As a result, we can reformulate the action space to consist of individual items, $A = |Z|$. In expectation, maximizing the novelty and relevance reward function for $j \in [1, k]$ separately would imply maximizing the top-k reward.

While the action space is reduced in size, a key benefit of our formulation is that the reward can still be a function of the top-k retrieved items from some model. This is because the environment can decide the reward based on whether the item is a part of the top-k items for the query. Recall that, given a state (query) x and an action (item) $z \in Z$, the novelty reward is dependent on whether the z is part of the top-k items returned by the base model ψ. As an example, we provide a simple reward function combining item-wise relevance and novelty w.r.t. ψ. Given any query x and item z, and a relevance oracle, `Rel` (i.e., an LLM), the reward is given by,

$$\mathcal{R}_b(x,z) = \begin{cases} 1 & \text{if } \mathrm{Rel}(x,z) = 1 \text{ and } z \notin topk(x,\psi) \\ -1 & \text{if } \mathrm{Rel}(x,z) = 0 \text{ and } z \in topk(x,\psi) \\ -0.5 & \text{if } \mathrm{Rel}(x,z) = 1 \text{ and } z \in topk(x,\psi) \\ -1 & \text{if } \mathrm{Rel}(x,z) = 0 \text{ and } z \notin topk(x,\psi) \end{cases} \tag{6}$$

We can see that the reward function penalizes irrelevant items and relevant items that are not novel w.r.t. the base model, while encouraging relevant items that are novel. However, in practice, the LLM-based relevance function may be noisy. In particular, as higher relevance items are likely to be sampled more, false negatives can be a problem. Therefore, for noisy oracles, we propose a simpler reward function that is only activated for relevant items but not for irrelevant items predicted by the oracle.

$$\mathcal{R}_b(x,z) = \begin{cases} 1 & \text{if } \mathrm{Rel}(x,z) = 1 \text{ and } z \notin topk(x,\psi) \\ -0.5 & \text{if } \mathrm{Rel}(x,z) = 1 \text{ and } z \in topk(x,\psi) \\ 0 & \text{otherwise} \end{cases} \tag{7}$$

4.2 Reduction to binary action space

To further improve the convergence rate, we consider a different RL formulation where the state is ⟨query, item⟩ pair and the policy outputs the probability of selecting the item. The item-wise reward function changes slightly to accommodate the two actions of selecting the item (1) or not (0). Assuming the same reward logic from Eqn. 6, the reward function becomes, $\mathcal{R}((x,z),a) = a\mathcal{R}_b(x,z) + (1-a)(-\mathcal{R}_b(x,z))$. Intuitively, if the policy selects the item for a query, then its reward for the action is proportional to $\mathcal{R}_b$. Otherwise, if it does not select the item, then its reward is proportional to negative of $\mathcal{R}_b$. The corresponding gradient is,

$$\nabla \mathbf{E}_{x,z\sim\mathcal{D}}\mathbf{E}_{\mathbf{a}\sim\pi'_\theta(x,z)}\mathcal{R}((x,z),a) \tag{8}$$

$$\approx \frac{1}{B}\sum_{i=1}^{B}[a^{(i)}\mathcal{R}_b(x^{(i)},z^{(i)}) - (1-a^{(i)})\mathcal{R}_b(x^{(i)},z^{(i)})]\nabla \log \pi'_\theta(a^{(i)}|x^{(i)},z^{(i)})$$

where $\pi'_\theta(a=1|x,z) = \pi_\theta(x,z) = \mathrm{softmax}_Z\ \mathtt{sim}(f_\theta(x), f_\theta(z))$ and $\pi'_\theta(a=0|x,z) = 1 - \pi_\theta(x,z) = 1 - \mathrm{softmax}_Z\ \mathtt{sim}(f_\theta(x), f_\theta(z))$.

With this formulation, the number of states increases to SA but the number of actions reduces to 2. As we show below, the convergence rate is significantly faster since the error now grows linearly with A rather than quadratic.

Proposition 2. *With the new formulation, under the assumptions of Proposition 1, for all* $t \geq 1$,

$$\mathbf{E}_{x\sim\mathcal{D}}(\pi^*(x) - \pi_{\theta_t}(x))^T \mathbf{r}(x) \leq \frac{64SA\rho^2}{t} \left\| \frac{1}{\mu} \right\|_\infty \tag{9}$$

Note that in practice, ρ may be higher for the binary action formulation since there are only 2 actions. Assuming a "good enough" supervised policy, conservative value for ρ may be ~ 50, implying that probability of the optimal action under supervised policy is always $\geq 1/(2 \times 50) = 0.01$. Even under this conservative estimate, as long as the number of actions is of the order of millions, $\rho^2 << A$ and hence the convergence rate in Proposition 2 would be significantly faster. In other words, as long as $\rho^2 << A$, the binary-action policy gradient algorithm will converge at a faster rate.

For maximizing a reward based on novelty and relevance such as Eqn. 6, the ρ parameter in Proposition 2 also implies that using the base policy as the initial policy may not be the best choice. The base policy is expected to be significantly better than a random policy at relevance, so it will assign higher than random probability to relevant items—both novel and not novel items. However, within relevant items, by definition, its probability for novel items will be lower than that of the not novel items. To increase the probability of the optimal action under the initial policy even further (and decrease ρ), we can use additional training data to finetune the base policy using supervised learning (e.g., InfoNCE loss from Eq. 1). We call this model the *Supervised Model*. As long as the relevance

accuracy does not decrease, randomness in the training procedure should shift the order of top-k items under the supervised model. As a result, the supervised model may retain the property of being better than a random policy at relevance (just like the base policy) but may also yield higher probability to some novel items under the base policy. Therefore, whenever additional training data is available, such a supervised model is expected to have a lower ρ and we recommend to use it as the initial policy.

4.3 Proposed algorithm: PG-Ret

The above discussion indicates the following conditions for a fast convergence rate with policy gradient, **1)** the action space should be small; and **2)** the initial policy should be as close to the optimal as possible. We addressed the first condition through a binary-action formulation of the problem. For the second condition, we proposed initializing the policy optimization with a policy trained using supervised learning (i.e., using Eq. 1).

Algorithm 1 shows the resultant training algorithm. In each batch, B query-item pairs (states) are sampled and the reward is computed for each state. Using Eqn. 8, we compute the gradient and update the policy parameters θ at the end of the batch. For each application, one needs to specify the sampling procedure for query-item pairs, the reward function, and the estimation procedure for $\pi'(a|x,z)$.

Sampling query-item states. Since items are also a part of the state in our proposed formulation, a key question is how to sample the states $\langle x, z\rangle$. To map to the original policy gradient formulation, we can sample queries x randomly from the train dataset for each batch. For each query, we compute the similarity scores with all items using the encoder f_θ corresponding to the current policy and then sample items proportional to their score. Note that we are not restricted to only sampling items proportional to their similarity score (since items are not actions now). Therefore, we also add exploration by sampling items from another retrieval model (trained independently). For example, for novelty, a natural choice is to sample from predictions of the base model, restricting the sampling only to items that are ranked beyond top-L. Such items will be novel by definition and thus we only need to check for relevance. More generally, we may use any pretrained encoder suitable for the task. Finally, there is a risk that the policy overfits to the reward from the relevance oracle and "forgets" the true user feedback data on which the supervised policy was trained [26]. Thus, we should also add the user feedback data $(x, z) \in D$ during training. To this end, in practice, all three sources are combined stochastically: we sample a query x randomly from the dataset and then sample either $z \sim \pi_{\theta_t}(x)$ with probability α; $z \sim \pi_{\theta explore_t}(x)$ with probability β; or $z \sim P_{\mathcal{D}}(x)$ with probability $1 - \alpha - \beta$.

Estimating $\pi'(a|x,z)$. While $\pi'(a|x,z)$ is defined as a softmax operation, computing the softmax over all items is a compute-intensive procedure. Moreover, training procedures for recommendation systems have benefitted from using contrastive losses. Therefore, we we implement an approximation of the softmax using contrastive losses. A straightforward approximation is to only use the items in the current batch as negatives (random in-batch negatives). However, since we initialize with a well-trained supervised policy, we may find that most of the contrastive losses are zero since the chosen item is already closer to the query than random negatives. To speed up training, we use negatives that are aimed at optimizing novelty. Specifically, for each query, we use the top-M items returned by the base model as negatives. M is a hyperparameter; too high M may destroy all relevance. Therefore, we define two kinds of losses: **1)** *Aggressive-Novelty:* InfoNCE loss from Eqn. 1 with top-M negatives (typically M is small, e.g., $M = 5$); **2)** *Conservative-Novelty:* Triplet loss [17], bounded $\in [0, 1]$ and margin=0 (effectively $M = 1$).

Efficiency considerations. Note that to compute top-k items or sample items for a given query, the entire set of actions have to be encoded by the current encoder f_θ. For computationally efficiency, we fix the item encoder to be the initial encoder and only update the query encoder. This avoids having to recompute the item embeddings for each query in the batch. That is, only the query encoder is updated during policy optimization.

Algorithm 1 Large action PG

Input: Initial Policy π_θ, Base Model ψ, Training dataset D, Relevance Oracle `rel`, Number of epochs N, Batch size B, Learning rate η, $\alpha, \beta \in [0, 1]$
Output: Trained policy $\hat{\pi}_\theta$.
for epoch=1,2,3..N **do**
  for $D_{batch} \sim D$ **do**
    $L = 0$
    $i = 0$
    $(x, z) \sim D$
    while $i < B$ **do**
      $a \sim \pi'_\theta(x, z)$ // Sample Action
      $r_b = \mathcal{R}_b(x, z, \texttt{topk}(x, \psi), \texttt{rel}(x, z))$ // Reward depends on top-k items and relevance oracle
      $r = r_b a + (-r_b)(1 - a)$
      $L = L - r \log \pi'_\theta(a|x, z)$
      $i = i + 1$
      $x \sim D$
      $z \sim \pi_\theta(x)(\text{wProb } \alpha), \sim \pi_{\theta explore}(x)(\text{wProb } \beta)$, or $\sim D(\text{wProb } 1 - \alpha - \beta)$
    end while
    $\theta = \theta - \eta \nabla L$ // Can use any gradient optimizer
  end for
end for

5 EVALUATION

We evaluate PG-Ret on increasing novelty of the top-k items of a retrieval model compared to an existing model's output.

5.1 Setup: Datasets, Metrics, and Baselines

Setup. We consider the production setup for a recommendation system, wherein there is an existing retrieval algorithm in production. We call this model the *base* model. Typically, this model is trained on millions of query-item clicked pairs collected from log data. We assume access to a small amount of new training data (e.g., query-item pairs collected from log data after the base model has been deployed) and evaluate different ways to produce a new retrieval model with high novelty compared to the base model. We